

```

// Structural description of a 4-bit universal shift register (see Fig. 6.7)
module Shift_Register_4_str (           // V2001, 2005
    output [3: 0]    A_par,             /   / Parallel output
    input [3: 0]     I_par,             // Parallel input
    input            s1, s0,           // Mode select
    input            MSB_in, LSB_in, CLK, Clear_b // Serial inputs, clock,
    clear
);
// bus for mode control
    wire [1:0] select = {s1, s0};
// Instantiate the four stages
    stage ST0 (A_par[0], A_par[1], LSB_in, I_par[0], A_par[0], select, CLK, Clear_b);
    stage ST1 (A_par[1], A_par[2], A_par[0], I_par[1], A_par[1], select, CLK, Clear_b);
    stage ST2 (A_par[2], A_par[3], A_par[1], I_par[2], A_par[2], select, CLK, Clear_b);
    stage ST3 (A_par[3], MSB_in, A_par[2], I_par[3], A_par[3], select, CLK, Clear_b);
endmodule

// One stage of shift register
module stage (i0, i1, i2, i3, Q, select, CLK, Clr_b);
    input            i0, // circulation bit selection
    i1,              // data from left neighbor or serial input for shift-right
    i2,              // data from right neighbor or serial input for shift-left
    i3;              // data from parallel input
    output           Q;
    input [1: 0]     select; // stage mode control bus
    input            CLK, Clr_b; // Clock, Clear for flip-flops
    wire             mux_out;
// instantiate mux and flip-flop
    wire Clr = ~Clr_b // Flip-flop has active-high clear signal
// but circuit has
    active-low clear action
    Mux_4x1 M0 (mux_out, i0, i1, i2, i3, select);
    D_flip_flop M1 (Q, mux_out, CLK, Clr);
endmodule

// 4x1 multiplexer // behavioral model
module Mux_4x1 (mux_out, i0, i1, i2, i3, select);
    output mux_out;
    input i0, i1, i2, i3;
    input [1: 0] select;
    reg mux_out;
    always @ (select, i0, i1, i2, i3)
        case (select)
            2'b00: mux_out = i0;
            2'b01: mux_out = i1;
            2'b10: mux_out = i2;
            2'b11: mux_out = i3;
        endcase
endmodule

```

```
// Behavioral model of D flip-flop
module D_flip_flop (Q, D, CLK, Clr);
  output    Q;
  input     D, CLK, Clr;
  reg Q;
  always @ (posedge CLK, posedge Clr)
    if (Clr) Q <= 1'b0; else Q <= D;
endmodule
```
